UNIT - V

Database Connectivity with MySql
• Introduction of Unit • Different methods of database connectivity • Creating a MySql Database • Connection with MySql Database • Project • Conclusion of Unit

Introduction of Unit

This Unit focuses on database connectivity with MySQL, one of the most widely used relational database management systems (RDBMS) in the industry. In this unit, we will learn how to establish connections between software applications and MySQL databases, enabling continuous interaction and data manipulation.

The unit begins with an overview of database connectivity concepts, emphasizing the importance of integrating databases into software systems for efficient data storage and retrieval. We will explore the fundamental principles of MySQL databases, including their structure, tables, and relationships. Throughout the unit, practical hands-on exercises and examples will guide us in implementing database connectivity solutions using MySQL connectors and APIs.

By the end of the unit, we will have a comprehensive understanding of database connectivity techniques with MySQL, empowering to develop robust and scalable software applications that leverage the power of relational databases for data management.

DBMS V/s. RDBMS

DBMS (Database Management System) and RDBMS (Relational Database Management System) are both types of software systems used to manage databases, but they differ in their underlying data models and functionality.

Comparison between DBMS and RDBMS:

1. **Data Model:**
    - **DBMS:** DBMS can support various types of data models, including hierarchical, network, and object-oriented models. These models organize data in different structures, such as trees, graphs, or objects.
    - **RDBMS:** RDBMS strictly follows the relational data model, where data is organized into tables consisting of rows and columns. Each table represents an entity, and relationships between entities are established using foreign keys.

2. **Data Integrity:**
    - **DBMS:** DBMS may or may not enforce data integrity constraints, such as referential integrity and entity integrity, depending on the specific implementation.
    - **RDBMS:** RDBMS enforces strict data integrity constraints to ensure the accuracy and consistency of data. This includes primary key constraints, foreign key constraints, unique constraints, and check constraints.

3. **Data Manipulation:**
    - **DBMS:** DBMS provides basic data manipulation capabilities, such as adding, modifying, deleting, and retrieving data from databases.
    - **RDBMS:** RDBMS supports complex data manipulation operations using SQL (Structured Query Language), including SELECT (to retrieve data),

INSERT (to add new data), UPDATE (to modify existing data), and DELETE (to remove data). It also supports advanced SQL features like joins, subqueries, and aggregations.

4. **Query Language Support:**
    - **DBMS:** DBMS may support query languages specific to the data model used, which may vary depending on the vendor.
    - **RDBMS:** RDBMS universally supports SQL as the standard query language for interacting with databases. SQL provides a standardized syntax for querying and manipulating relational data.

5. **Normalization:**
    - **DBMS:** DBMS may or may not support normalization, which is the process of organizing data to minimize redundancy and dependency.
    - **RDBMS:** RDBMS fully supports normalization techniques to eliminate data redundancy and maintain data consistency. This involves decomposing tables into smaller, well-structured entities to reduce anomalies and improve database efficiency.

6. **ACID Properties:**
    - **DBMS:** DBMS may or may not guarantee ACID properties (Atomicity, Consistency, Isolation, Durability) for transactions, depending on the specific implementation.
    - **RDBMS:** RDBMS ensures ACID properties for transactions, ensuring that transactions are executed reliably and consistently, even in the presence of failures or concurrent access.

While both DBMS and RDBMS are used for managing databases, RDBMS specifically follows the relational data model, enforces data integrity constraints, supports SQL as the query language, and ensures ACID properties for transactions. These features make RDBMS well-suited for handling structured data in a wide range of applications.

Database connectivity

Database connectivity refers to the mechanisms and techniques used to establish communication between a software application and a database management system (DBMS). This connectivity enables the application to interact with the database, perform operations such as querying, updating, and managing data, and retrieve results for processing.

Concepts related to database connectivity:

1. **Connection Establishment:**
    - Database connectivity begins with establishing a connection between the application and the DBMS. This involves providing connection parameters such as the hostname, port number, database name, username, and password.
    - Connections can be established using different protocols, such as TCP/IP, JDBC (for Java applications), ODBC (for C/C++ applications), or ADO.NET (for .NET applications).

2. **Data Access Interfaces:**
    - Data access interfaces or APIs provide the programming interfaces through which applications can interact with the database.
    - Examples of data access interfaces include JDBC (for Java), ADO.NET (for .NET), PDO (for PHP), and DB-API (for Python).

3. **Connection Pooling:**
    - Connection pooling is a technique used to manage and reuse database connections to improve performance and resource utilization.
    - Instead of opening a new database connection every time an application needs to interact with the database, connection pooling maintains a pool of pre-established connections that can be reused by multiple client applications.

4. **SQL (Structured Query Language):**
    - SQL is the standard language used for querying and manipulating data in relational database management systems.
    - Applications use SQL commands to perform operations such as SELECT (to retrieve data), INSERT (to insert new records), UPDATE (to modify existing records), DELETE (to remove records), and CREATE/ALTER/DROP (to manage database objects).

5. **Transaction Management:**
    - Transaction management ensures the integrity and consistency of data by grouping multiple database operations into atomic units called transactions.
    - Transactions adhere to the ACID properties: Atomicity (all or nothing), Consistency (maintains data integrity), Isolation (operations are isolated from each other), and Durability (changes are persistent).

6. **Error Handling:**
    - Effective error handling is crucial in database connectivity to handle issues such as connection failures, SQL syntax errors, transaction rollbacks, and data integrity violations.

- Applications should implement mechanisms to gracefully handle errors, log diagnostic information, and provide feedback to users.

7. **Security:**
    - Database connectivity also involves ensuring the security of data and preventing unauthorized access or manipulation.
    - Security measures may include using encrypted connections (SSL/TLS), implementing authentication and authorization mechanisms, and applying access controls at the database level.

These database connectivity concepts helps to design, implement, and maintain robust and secure database-driven applications.

Importance of Integration of Database into Software Systems

Integrating databases into software systems is of paramount importance for various reasons,

1. **Data Storage and Management:** Databases serve as centralized repositories for storing structured data in a manner that is efficient, organized, and scalable. By integrating databases into software systems, developers can securely store and manage vast amounts of data, ensuring its accessibility and reliability.
2. **Data Integrity and Consistency:** Database management systems (DBMS) enforce data integrity rules and maintain consistency, ensuring that stored data remains accurate and reliable over time. Integrating databases into software systems helps enforce these rules, preventing data corruption, duplication, or loss.
3. **Data Retrieval and Analysis:** Software systems often require access to data for various purposes, such as generating reports, performing analytics, and making informed business decisions. Integrating databases allows applications to retrieve data efficiently using queries and filters, enabling real-time analysis and insights.
4. **Transaction Processing:** Many software systems involve transactional operations that require atomicity, consistency, isolation, and durability (ACID) properties. Databases support transaction processing by grouping multiple database operations into atomic units, ensuring data integrity and reliability even in the event of system failures or interruptions.

5. **Concurrency Control:** In multi-user environments, database concurrency control mechanisms ensure that multiple users can access and modify data

simultaneously without conflicts or inconsistencies. By integrating databases, software systems can leverage concurrency control features to manage concurrent access to shared data resources effectively.

6. **Scalability and Performance:** Databases are designed to scale horizontally and vertically to accommodate growing data volumes and user loads. Integrating databases into software systems enables applications to scale seamlessly by leveraging features such as replication, sharding, and clustering to distribute data and workload across multiple servers.
7. **Security and Access Control:** Databases provide robust security features to protect sensitive data from unauthorized access, manipulation, or disclosure. By integrating databases, software systems can enforce access controls, implement encryption, and audit database activities to ensure compliance with security policies and regulations.
8. **Application Modularity and Reusability:** Separating data storage and management from application logic promotes modularity and reusability in software systems. By integrating databases, developers can design applications that are easier to maintain, update, and extend, as changes to the data model do not necessitate modifications to the entire application codebase.

Integrating databases into software systems is crucial for enabling efficient data storage, retrieval, management, and analysis, while ensuring data integrity, security, scalability, and performance. It allows applications to leverage the full capabilities of database management systems to meet the evolving needs of users and businesses.

Different Methods of Database Connectivity

Database connectivity refers to the ability of software applications to interact with databases, allowing for the retrieval, manipulation, and management of data stored

within the database management system (DBMS). There are several methods of establishing database connectivity, each with its own advantages, use cases, and implementation considerations.

Methods of database connectivity:

1. **Native Database Connectivity:** Native database connectivity **involves using native drivers or libraries provided by the database vendor to establish connections between the application and the database.** These drivers are specifically designed to interact with the target database management system and offer optimal performance and compatibility. Examples of native database connectivity methods include:
    - JDBC (Java Database Connectivity) for Java applications.
    - ODBC (Open Database Connectivity) for applications written in languages like C, C++, and Python.
    - ADO.NET (ActiveX Data Objects .NET) for .NET applications.
    - PDO (PHP Data Objects) for PHP applications. Native database connectivity is often preferred for its efficiency and direct integration with the target database system.
2. **ODBC (Open Database Connectivity):** ODBC is a standard API (Application Programming Interface) that allows applications to access databases using SQL queries. It provides a common interface for database connectivity across different platforms and programming languages. ODBC drivers act as intermediaries between the application and the database, translating ODBC function calls into native database calls. This approach offers portability and flexibility, allowing applications to switch between different database systems without significant code changes.

3. **OLE DB (Object Linking and Embedding Database):** OLE DB is a Microsoft data access technology that enables applications to access data from a variety of sources, including relational databases, spreadsheets, and text files. OLE DB providers serve as intermediaries between the application and the data source, abstracting the underlying data access mechanisms. While OLE DB offers greater flexibility than ODBC in terms of data source types, it has largely been superseded by newer technologies like ADO.NET.

4. **ADO.NET (ActiveX Data Objects .NET):** ADO.NET is a set of classes and libraries in the .NET Framework that provides data access services for .NET applications. It includes the **SqlConnection** and **SqlCommand** classes for connecting to and executing commands against SQL Server databases. ADO.NET supports both connected and disconnected data access models, allowing for efficient data retrieval and manipulation.

5. **ORM (Object-Relational Mapping):** ORM frameworks like Hibernate (for Java), Entity Framework (for .NET), and **SQLAlchemy** (for Python) provide a higher-level abstraction for database connectivity by mapping database tables to object-oriented programming constructs. Developers interact with the database using objects and classes, rather than writing SQL queries directly. ORM frameworks handle the translation of object operations to SQL statements, simplifying database interaction and reducing the amount of boilerplate code required.

6. **RESTful APIs (Representational State Transfer):** RESTful APIs provide a

lightweight and platform-independent method for accessing and manipulating data stored in databases. Using HTTP methods (GET, POST, PUT, DELETE) and standard data formats like JSON or XML, applications can perform CRUD operations (Create, Read, Update, Delete) on database resources. RESTful APIs offer simplicity, scalability, and compatibility with a wide range of programming languages and platforms.

These are some of the common methods of establishing database connectivity in software applications. **The choice of method depends on factors such as the programming language, the database management system, performance requirements, and the complexity of the application's data access needs.**

Introduction to MySQL

MySQL is an open-source relational database management system (RDBMS) that is widely used for managing and organizing structured data. Developed by MySQL AB (now owned by Oracle Corporation), MySQL is known for its reliability, scalability, and ease of use, making it a popular choice for both small-scale applications and large enterprise systems.

Features of MySQL include:

1. **Relational Database Management System (RDBMS):** MySQL follows the relational model, allowing data to be stored in tables with rows and columns. It supports SQL (Structured Query Language) for querying and manipulating data.
2. **Open Source:** MySQL is open-source software, which means it is freely available for use, modification, and distribution under the terms of the GNU General Public License (GPL). This makes it accessible to a wide range of developers and organizations.
3. **Cross-Platform Compatibility:** MySQL is available for various operating systems, including Linux, Windows, macOS, and Unix-like systems. It provides consistent performance and functionality across different platforms.
4. **Scalability:** MySQL is designed to scale from small, single-user applications to large, high-traffic websites and enterprise-level systems. It supports features like replication, clustering, and sharding to handle increasing data volumes and user loads.

5. **Performance Optimization:** MySQL offers various optimization techniques to improve database performance, such as indexing, query caching, and storage

engine options. It also provides tools for monitoring and tuning database performance.

6. **High Availability and Reliability:** MySQL supports features like replication and clustering to ensure high availability and fault tolerance. It allows for the creation of backup copies and automated failover mechanisms to minimize downtime.
7. **Security:** MySQL offers robust security features to protect sensitive data, including access controls, encryption, and authentication mechanisms. It supports SSL/TLS for secure connections and integrates with external authentication systems like LDAP and Kerberos.
8. **Community Support:** MySQL has a large and active community of developers, users, and contributors who provide support, share knowledge, and contribute to the ongoing development of the software. There are numerous resources available, including documentation, forums, and user groups.

MySQL is a powerful and versatile database management system that is suitable for a wide range of applications, from simple websites and blogs to complex enterprise solutions. Its combination of performance, scalability, reliability, and ease of use makes it a popular choice for developers and organizations worldwide.

PHP-MySQL Database Introduction

MySQL is an open-source relational database management system (RDBMS). It is the most popular database system used with PHP. MySQL is developed, distributed, and supported by Oracle Corporation.

- The data in a MySQL database are stored in tables which consists of columns and rows.
- MySQL is a database system that runs on a server.
- MySQL is ideal for both small and large applications.
- MySQL is very fast, reliable, and easy to use database system. It uses standard SQL

How to connect PHP with MySQL Database?

PHP 5 and later can work with a MySQL database using:

1. MySQLi extension.
2. PDO (PHP Data Objects).

Difference Between MySQLi and PDO

- PDO works on 12 different database systems, whereas MySQLi works only with MySQL databases.
- Both PDO and MySQLi are object-oriented, but MySQLi also offers a procedural API.

Connecting to MySQL database using PHP

There are three ways of working with MySQl and PHP

1. MySQLi (object-oriented)
2. MySQLi (procedural)
3. PDO

- **Using MySQLi procedural procedure** : There is also a procedural approach of MySQLi to establish a connection to MySQL database from a PHP script as described below.

Syntax:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Creating connection
$conn = mysqli_connect($servername, $username, $password);
```

```
// Checking connection
if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Output: Connected successfully

Explanation: In MySQLi procedural approach instead of creating an instance we can use the mysqli_connect() function available in PHP to establish a connection. This function takes the information as arguments such as host, username , password , database name etc. This function returns MySQL link identifier on successful connection or FALSE when failed to establish a connection.

PHP - MySQL (Creating Database)

The basic steps to create MySQL database using PHP are:

- Establish a connection to MySQL server from your PHP script.
- If the connection is successful, write a SQL query to create a database and store it in a string variable.
- Execute the query.

Using MySQLi Procedural procedure: If the MySQL connection is established using procedural procedure then we can use the mysqli_query() function of PHP to execute our query as described in the below syntax.

Syntax:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Creating connection
$conn = mysqli_connect($servername, $username, $password);
// Checking connection
if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}
// Creating a database named newDB
$sql = "CREATE DATABASE newDB";
if (mysqli_query($conn, $sql)) {
```

```
    echo "Database created successfully with the name newDB";
} else {
    echo "Error creating database: " . mysqli_error($conn);
}

// closing connection
mysqli_close($conn);
?>
```

Output: Database created successfully with the name newDB

PHP Database connection

XAMPP stands for cross-platform, Apache, MySQL, PHP, and Perl. It is among the simple light-weight local servers for website development.

Requirements: XAMPP web server procedure:

- Start XAMPP server by starting Apache and MySQL.
- Write PHP script for connecting to XAMPP.
- Run it in the local browser.
- Database is successfully created which is based on the PHP code.

In PHP, we can connect to the database using XAMPP web server by using the following path. "localhost/phpmyadmin"

Steps in Detail:

- Open XAMPP and start running Apache, MySQL

Now open your PHP file and write your PHP code to create database and a table in your database.

PHP code to create a database:

```
<?php
// Server name must be localhost
$servername = "localhost";
```

```
// In this case, user name will be root
$username = "root";

// Password is empty
$password = "";

// Creating a connection
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
        die("Connection failure: ".
        $conn->connect_error);
}

// Creating a database named student
$sql = "CREATE DATABASE student";if
($conn->query($sql) === TRUE) {
        echo "Database with name student";
} else {
        echo "Error: " . $conn->error;
}

// Closing connection
$conn->close();
?>
```

- Save the file as “data.php” in *htdocs* folder under XAMPP folder
- Then open your web browser and type *localhost/data.php*

Finally the database is created and connected to PHP.

Connect PHP to MySQL

Connection to MySQL using MySQLi

- PHP provides mysql_connect() function to open a database connection.
- This function takes a single parameter, which is a connection returned by the mysql_connect()function.
- We can disconnect from the MySQL database anytime using another PHP function mysql_close().

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Connection
$conn = mysqli_connect($servername,
                       $username, $password);

// Check if connection is
// Successful or not
if (!$conn) {
die("Connection failed: "
        . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

PHP | Inserting into MySQL database

INSERT INTO statement is used to insert new rows in a database table. Let's see the syntax how to insert into table, considering database already exists.

SYNTAX : INSERT INTO TABLE_NAME (column1, column2, column3, ... columnN) VALUES (value1, value2, value3, ...valueN);

So to create a SQL query using the INSERT INTO statement with appropriate values, here's an example, which will insert a new row to the newDB table by specifying values for the first_name, last_name and email fields.

```
<?php
$link = mysqli_connect("localhost", "root", "", "newdb");

if ($link == = false) {
        die("ERROR: Could not connect. ".mysqli_connect_error());
}

$sql = "INSERT INTO mytable (first_name, last_name, age)
                VALUES('ram', 'singh', '25') ";
```

```
	if (mysqli_query($link, $sql))
{
	echo "Records inserted successfully.";
}
else
{
	echo "ERROR: Could not able to execute $sql. "
		.mysqli_error($link);
}


mysqli_close($link);
? >
```

Output : Records inserted successfully

PHP | MySQL Select Query

The SQL **SELECT** statement is used to select the records from database tables.

```
<?php
$link = mysqli_connect("localhost", "root", "", "Mydb");

if ($link === false) {
	die("ERROR: Could not connect. "
					.mysqli_connect_error());
}

$sql = "SELECT * FROM Data";
if ($res = mysqli_query($link, $sql)) {
	if (mysqli_num_rows($res) > 0) {
		echo "<table>";
		echo "<tr>";
		echo "<th>Firstname</th>";
```

```
                echo "<th>Lastname</th>";
                echo "<th>age</th>";
                echo "</tr>";
                while ($row = mysqli_fetch_array($res)) {
                        echo "<tr>";
                        echo "<td>".$row['Firstname']."</td>";
                        echo "<td>".$row['Lastname']."</td>";
                        echo "<td>".$row['Age']."</td>";
                        echo "</tr>";
                }
                echo "</table>";
                mysqli_free_result($res);
        }
        else {
                echo "No matching records are found.";
        }
}
else {

        echo "ERROR: Could not able to execute $sql. "
                                                        .mysqli_error($link);
}
```

```
mysqli_close($link);
?>
```

Code Explanation:

1. The "res" variable stores the data that is returned by the function *mysql_query()*.
2. Everytime *mysqli_fetch_array()* is invoked, it returns the next row from the *res()* set.
3. The while loop is used to loop through all the rows of the table "data".